

ANNA UNIVERSITY, CHENNAI – 600 025

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

ST. ANNE'S COLLEGE OF ENGINEERING AND TECHNOLOGY, PANRUTI

NAAN MUDHALVAN COURSE REPORT

ON

INDUSTRIAL IoT AND INDUSTRY 4.0

Course Code: CEC367

Course Title: Industrial IoT and Industry 4.0

Semester: V

Academic Year: 2025 – 2026

Prepared By:

B. DEVI SRI

Trainer –Mr.S. Prabu

Submitted By:

Name: _____

Register No.: _____

Department: _____

College: _____

Under the Guidance of:

Mr. B. Arunkumar - Assistant professor

Department of Electronics and communication Engineering

CERTIFICATE

This is to certify that this is the bonafide record of the “Industrial IoT and Industry 4.0” laboratory work done by

Name: _____

Register No: _____

Department: _____

College: _____

During the academic year 2025 – 2026 in partial fulfillment of the requirements for the Naan Mudhalvan Skill Development Program, conducted in association with Anna University and InGage Technologies.

Staff In-charge

Head of the Department

External Examiner

ACKNOWLEDGEMENT

We express our heartfelt gratitude to our beloved Secretary Rev. Sr. Punitha jilt and respected principal Dr. R. Arokiadass, for providing the opportunity to carry out this lab work under the Naan Mudhalvan initiative.

We extend our sincere thanks to Rev.Sr.Dr.S.Anita, Head of the Department ,Depatment of Electronics and Communication Engineering, for her valuable guidance and support.

We also thank InGage Technologies for designing and conducting the Industrial IoT and Industry 4.0 course, which helped us gain hands-on skills in IoT, AI, Cloud, and Digital Twin technologies.

Finally, we thank our trainer, Mr.S.Prabu and Course Incharge, Mr. B. Arun kumar for their constant encouragement, mentorship, and technical assistance throughout the course.

Student Signature _____

Date: _____

TABLE OF CONTENTS

S. No.	Experiment Title	Page No.
1	Immersive AI-Driven Factories – Predictive Maintenance	5
2	AI-Powered Smart Farming – Precision Analytics	6
3	Intelligent Oil Rig Systems – Failure Forecasting	7
4	Cold Chain Optimization – Spoilage Prediction	8
5	AI Quality Inspection – Defect Detection	9
6	Smart Energy Grids – Load Forecasting	10
7	Remote Patient Monitoring – AI-Based Insights	11
8	Warehouse Optimization – Predictive Inventory	12
9	Water Infrastructure Safety – Leak Detection	13
10	Fleet Efficiency Solutions – Route Optimization	14
11	AI Debug Operations – Control Room Dashboards	15
12	Mining Safety Systems – Hazard Forecasting	16
13	Smart Building HVAC – Energy Load Prediction	17
14	Livestock Health Analytics – Disease Prediction	18
15	Intelligent Parking Systems – Occupancy Forecasting	19
16	Predictive Inventory Management – Demand Forecasting	20
17	Fire & Gas Plant Safety – Risk Predictions	21
18	Precision Irrigation – Water Usage Forecasts	22
19	Warehouse Robotics – Path Optimization	23
20	AI-Driven Supply Chains – Risk Visualization	24

Experiment 1: Immersive AI-Driven Factories – Predictive Maintenance and Digital Twin Visualization

Aim:

To develop a real-time industrial machine monitoring system using AI for predictive maintenance and visualize it as a digital twin in Unity.

Apparatus / Software Required:

- Unity
- Wokwi (for IoT simulation)
- MQTT Broker (HiveMQ or Mosquitto)
- ThingSpeak (for cloud data logging)
- Google Colab (for ML model training)
- Power BI (for analytics dashboards)
- Convai (for conversational AI)

Theory:

Industrial IoT (IIoT) connects sensors, machines, and analytics to improve manufacturing efficiency. Predictive maintenance uses AI models to forecast machine failures before they occur. A *digital twin* in Unity acts as a virtual representation of a real factory, showing live sensor data and AI insights in 3D.

Procedure:

1. Create a machine setup in **Wokwi** with temperature, vibration, and speed sensors.
2. Connect the sensors to a **MQTT broker** and publish real-time data.
3. Log sensor data to **ThingSpeak** for cloud storage and visualization.
4. Train a **Machine Learning model** in Colab to predict failure probabilities.
5. Integrate live data and predictions into **Unity**, creating a digital factory environment.
6. Use **Power BI** to visualize machine health dashboards.
7. Add **Convai AI** to query machine status using voice commands.

Tasks / Code Summary:

- Sensor simulation (Wokwi)
- MQTT data streaming
- ML model: Predictive maintenance (failure prediction)
- Unity 3D visualization (digital twin)
- Convai integration (voice query)

Output / Expected Result:

- Real-time monitoring of machine status in Unity.
- AI model predicts potential failures.
- Voice-based status retrieval using Convai.

Result:

Successfully simulated an AI-driven industrial factory environment with real-time monitoring and predictive maintenance using digital twin visualization.

Experiment 2: AI-Powered Smart Farming – Precision Analytics and 3D Crop Health Monitoring

Aim:

To implement an AI-based smart farming system for monitoring soil and climate parameters, analyzing crop health, and visualizing the data in a 3D environment using Unity.

Apparatus / Software Required:

- Unity
- Wokwi (IoT simulator)
- MQTT Broker (HiveMQ or Mosquitto)
- ThingSpeak (cloud platform)
- Google Colab (for ML analytics)
- Power BI (for KPI dashboards)
- Convai (for voice-based interaction)

Theory:

Smart farming uses IoT sensors and artificial intelligence to increase agricultural productivity and efficiency. Sensors measure parameters like soil moisture, temperature, humidity, and sunlight. Machine learning models analyze these readings to predict irrigation needs and detect crop stress. Visualization in Unity provides a 3D farm view for real-time monitoring.

Procedure:

1. Create a virtual farm in **Unity** with crop fields, soil sensors, and weather stations.
2. Simulate soil moisture, humidity, and temperature sensors in **Wokwi**.
3. Stream sensor data to the **ThingSpeak** cloud using the **MQTT protocol**.
4. In **Google Colab**, train an ML model to analyze soil and weather data for irrigation recommendations.
5. Integrate Unity with ThingSpeak to visualize live soil and weather conditions.
6. Use **Power BI** to show yield predictions and farm KPIs.
7. Add **Convai** for voice queries like “How is the crop health today?”

Tasks / Code Summary:

- Sensor simulation for soil and climate
- MQTT communication setup
- ML model for irrigation and crop health analysis
- Unity visualization of 3D farm
- Power BI dashboards and Convai integration

Output / Expected Result:

- Real-time soil and weather data visualization in Unity.
- AI-based predictions for crop health and irrigation.
- Voice-based insights on crop condition.

Result:

Successfully implemented a smart farming system that combines IoT sensing, AI analytics, and 3D visualization for efficient agricultural monitoring.

Experiment 3: Intelligent Oil Rig Systems – Failure Forecasting and Offshore Digital Twin

Aim:

To design an intelligent oil rig monitoring system using IoT sensors, cloud communication, and AI models for failure forecasting, visualized through a digital twin in Unity.

Apparatus / Software Required:

- Unity
- Wokwi (for industrial sensor simulation)
- MQTT Broker (HiveMQ or Mosquitto)
- ThingSpeak (for cloud data storage)
- Google Colab (for AI model training)
- Power BI (for visualization and analytics)

Theory:

Offshore oil rigs operate under extreme conditions where equipment reliability is critical. IoT sensors can monitor temperature, vibration, and pressure in real time. AI models predict potential failures by analyzing sensor data trends. A digital twin of the rig in Unity provides an immersive, interactive visualization for maintenance and safety management.

Procedure:

1. Simulate vibration, temperature, and pressure sensors for rig machinery in **Wokwi**.
2. Send real-time sensor readings to the **MQTT broker** and forward them to **ThingSpeak**.
3. Use **Google Colab** to train a machine learning model that forecasts equipment faults.
4. Create a 3D **Unity** model of the oil rig with dynamic components (e.g., drilling arms, control panels).
5. Integrate live data from ThingSpeak into Unity using APIs for visualization.
6. Display analytical dashboards using **Power BI** to show rig health and predictions.
7. Set threshold-based alerts for abnormal readings within Unity visualization.

Tasks / Code Summary:

- Sensor data generation (temperature, vibration, pressure)
- MQTT → ThingSpeak → Unity data pipeline
- ML fault prediction model (Colab)
- 3D oil rig visualization and alert animation (Unity)
- Power BI dashboard linking

Output / Expected Result:

- Real-time oil rig data visualization in Unity.
- Predictive AI model highlights potential equipment failures.
- Power BI shows live operational metrics.

Result:

Successfully created an offshore oil rig digital twin that predicts equipment faults and visualizes real-time operations using IoT and AI.

Experiment 4: Cold Chain Optimization – AI Spoilage Prediction and Logistics Visualization

Aim:

To implement a cold chain monitoring system that predicts product spoilage using AI and visualizes live warehouse and logistics data in a 3D environment using Unity.

Apparatus / Software Required:

- Unity
- Wokwi (for IoT simulation)
- MQTT Broker (HiveMQ or Mosquitto)
- ThingSpeak (for cloud storage)
- Google Colab (for AI spoilage prediction model)
- Power BI (for analytics dashboards)

Theory:

A *cold chain* is a temperature-controlled supply chain used in food, medicine, and perishable logistics. Maintaining proper temperature throughout storage and transportation is critical. IoT sensors monitor environmental parameters, while AI predicts spoilage risks. Unity's 3D visualization helps track storage facilities and vehicles in real time, improving efficiency and reducing losses.

Procedure:

1. Simulate temperature and humidity sensors in **Wokwi** to represent warehouse and truck environments.
2. Stream sensor data through **MQTT** to **ThingSpeak** for cloud monitoring.
3. In **Google Colab**, train a regression model to predict spoilage risk based on temperature variations.
4. Design a 3D **Unity** warehouse and truck model.
5. Integrate live sensor data into Unity using ThingSpeak APIs for real-time display.
6. Highlight shipments exceeding safe temperature thresholds.
7. Use **Power BI** to visualize spoilage trends and delivery performance.

Tasks / Code Summary:

- IoT temperature & humidity data simulation
- MQTT → ThingSpeak integration
- AI spoilage prediction model (Colab)
- 3D warehouse visualization (Unity)
- Power BI performance dashboards

Output / Expected Result:

- Real-time temperature and humidity visualization of the cold chain.
- AI model identifies spoilage risks in shipments.
- Unity highlights risky packages and routes.

Result:

Successfully implemented a cold chain monitoring system with AI-based spoilage prediction and live logistics visualization in Unity.

Experiment 5: AI Quality Inspection – Defect Detection and Interactive Production Line Models

Aim:

To develop an AI-based quality inspection system for identifying defective products on an automated production line and visualize the process interactively in Unity.

Apparatus / Software Required:

- Unity
- Google Colab (for AI/ML defect detection model)
- Power BI (for quality analytics)
- Dataset of product images (defective and non-defective samples)

Theory:

In modern manufacturing, **AI-powered visual inspection** enhances quality assurance by automatically detecting defects using image recognition models. Machine learning techniques like CNN (Convolutional Neural Networks) classify images as defective or non-defective. Unity visualizes the entire production line in 3D, enabling real-time feedback and highlighting defective units interactively.

Procedure:

1. Collect or use a sample dataset of product images (good and defective).
2. Train a **CNN model** in **Google Colab** to classify images as defective or not.
3. Export the trained model and integrate it with a **Unity** production line simulation.
4. Simulate product movement along the conveyor belt using Unity animations.
5. Implement a script that uses AI results to color defective products in **red** and normal products in **green**.
6. Create a **Power BI** dashboard showing defect percentage, production rate, and efficiency trends.
7. Run the simulation and observe AI-based detection in real-time visualization.

Tasks / Code Summary:

- CNN model training for defect detection (Colab)
- Unity animation of production line
- Script to change color/state based on AI result
- Power BI integration for quality metrics

Output / Expected Result:

- Automated defect detection system that identifies faulty products using AI.
- Unity displays real-time production flow with visual cues for defective units.
- Power BI shows analytics such as defect ratio and line efficiency.

Result:

Successfully implemented an AI-based defect detection system integrated with Unity for real-time visual quality monitoring.

Experiment 6: Smart Energy Grids – Load Forecasting and Real-Time Grid Visualization

Aim:

To design a smart energy grid monitoring system using IoT data and AI-based load forecasting, and visualize the real-time power distribution in Unity.

Apparatus / Software Required:

- Unity
- Wokwi (IoT simulator)
- MQTT Broker (HiveMQ or Mosquitto)
- ThingSpeak (for cloud data storage)
- Google Colab (for load forecasting model)
- Power BI (for grid analytics and dashboards)

Theory:

A **smart grid** integrates IoT and AI technologies to monitor, predict, and optimize energy distribution. IoT sensors measure parameters like voltage, current, and power factor from transformers and smart meters. AI-based forecasting models help predict energy demand patterns, enabling better load balancing and preventing outages. Unity provides a 3D digital twin of the city grid for real-time visualization.

Procedure:

1. Simulate voltage and load sensors in **Wokwi** to represent transformers and distribution lines.
2. Publish sensor data via **MQTT** and send it to **ThingSpeak** for cloud logging.
3. Use **Google Colab** to train a regression model for **load forecasting** based on historical data.
4. Design a 3D **Unity** city environment with grid components—transformers, poles, and buildings.
5. Fetch live sensor readings from ThingSpeak and display them in Unity using API calls.
6. Create visual indicators for overloaded or underloaded transformers.
7. Integrate **Power BI** to show energy usage, demand trends, and forecasting insights.

Tasks / Code Summary:

- IoT sensor data simulation (voltage, current)
- MQTT → ThingSpeak data flow
- AI load forecasting model (Colab)
- 3D visualization of power grid (Unity)
- Power BI dashboards for analytics

Output / Expected Result:

- Real-time visualization of energy grid operations in Unity.
- AI-based prediction of power demand spikes.
- Power BI dashboard showing live energy distribution and forecast accuracy.

Result:

Successfully designed a smart energy grid system capable of real-time monitoring, predictive load management, and 3D visualization.

Experiment 7: Remote Patient Monitoring – AI-Based Health Insights and Hospital Visualization

Aim:

To create an AI-enabled remote patient monitoring system that collects vital signs using IoT sensors, analyzes health data with AI, and visualizes patient conditions in a 3D hospital environment using Unity.

Apparatus / Software Required:

- Unity
- Wokwi (for patient sensor simulation)
- MQTT Broker (HiveMQ or Mosquitto)
- ThingSpeak (for cloud storage)
- Google Colab (for AI health analytics)
- Power BI (for dashboards and health reports)

Theory:

Remote Patient Monitoring (RPM) is a healthcare application of Industrial IoT where patient health data such as heart rate, temperature, and oxygen level are continuously tracked using sensors. The data is transmitted to cloud platforms for real-time analysis. AI models predict health anomalies and alert medical staff. Visualization in Unity allows immersive monitoring of hospital rooms and patient status.

Procedure:

1. Simulate patient sensors in **Wokwi** for temperature, heart rate, and oxygen level.
2. Send live readings via **MQTT** to **ThingSpeak** cloud channels.
3. Train an AI model in **Google Colab** to detect abnormal patterns indicating potential health issues.
4. Build a 3D **Unity** hospital environment showing patient beds and monitoring screens.
5. Fetch real-time data from ThingSpeak and map each patient's vitals above their beds.
6. Color code patients: green (normal), yellow (warning), and red (critical).
7. Create **Power BI** dashboards displaying patient statistics and AI predictions.

Tasks / Code Summary:

- IoT simulation for patient sensors
- MQTT → ThingSpeak integration
- AI anomaly detection model (Colab)
- Unity 3D hospital visualization
- Power BI health analytics

Output / Expected Result:

- Real-time visualization of patient health data in Unity.
- AI model detects anomalies and generates alerts.
- Power BI dashboard shows individual and overall hospital insights.

Result:

Successfully implemented a remote patient monitoring system integrating IoT, AI, and 3D visualization for continuous healthcare tracking.

Experiment 8: Warehouse Optimization – Predictive Inventory and 3D Storage Management

Aim:

To develop a predictive inventory management system using IoT sensors and AI analytics, visualized in a 3D warehouse environment using Unity.

Apparatus / Software Required:

- Unity
- Wokwi (for IoT simulation)
- MQTT Broker (HiveMQ or Mosquitto)
- ThingSpeak (for cloud storage)
- Google Colab (for predictive analytics model)
- Power BI (for inventory dashboards)

Theory:

Warehouse optimization is a crucial part of Industry 4.0 where IoT and AI work together to streamline storage, inventory control, and logistics. IoT sensors monitor inventory levels, environmental parameters, and storage conditions. AI models predict demand and identify shortages before they occur. Unity provides a 3D visualization of stock levels and movements for operational efficiency.

Procedure:

1. Simulate RFID or proximity sensors in **Wokwi** to detect item counts or bin levels.
2. Connect sensors to an **MQTT broker** to send data to **ThingSpeak** cloud channels.
3. Build an **AI model in Google Colab** to analyze inventory trends and predict restocking needs.
4. Design a 3D **Unity warehouse** with storage racks, moving goods, and forklifts.
5. Map live data from ThingSpeak to Unity for real-time stock visualization.
6. Color code storage areas—green (adequate), yellow (low), red (out of stock).
7. Display inventory analytics and restock alerts in **Power BI**.

Tasks / Code Summary:

- IoT inventory sensor simulation (RFID/proximity)
- MQTT → ThingSpeak integration
- Predictive analytics model (Colab)
- Unity 3D warehouse visualization
- Power BI restock dashboards

Output / Expected Result:

- Real-time visualization of inventory levels in a 3D warehouse.
- AI prediction of future stock requirements.
- Power BI dashboard showing warehouse efficiency and shortage alerts.

Result:

Successfully developed a predictive warehouse optimization system integrating IoT sensors, AI analytics, and 3D visualization for effective inventory control.

Experiment 9: Water Infrastructure Safety – Leak Detection and Urban Pipeline Visualization

Aim:

To implement an AI-assisted water infrastructure monitoring system that detects leaks and visualizes real-time pipeline conditions in a 3D urban environment using Unity.

Apparatus / Software Required:

- Unity
- Wokwi (for IoT simulation)
- MQTT Broker (HiveMQ or Mosquitto)
- ThingSpeak (for cloud monitoring)
- Google Colab (for AI leak prediction model)
- Power BI (for analytics dashboards)

Theory:

Modern urban water networks use IoT technology to ensure continuous and efficient water distribution. Sensors placed in pipelines measure flow, pressure, and vibration to detect leakages. AI models analyze these parameters to predict possible leaks before major losses occur. Unity-based visualization enables maintenance teams to explore the city pipeline in 3D for faster response and prevention.

Procedure:

1. Simulate **flow and pressure sensors** in **Wokwi** to represent water pipelines.
2. Publish sensor readings via **MQTT** and send them to **ThingSpeak** for cloud storage.
3. In **Google Colab**, develop a **machine learning model** to predict or classify potential leaks.
4. Build a **3D Unity city model** with underground water pipelines.
5. Integrate live data from ThingSpeak to Unity for visual display of flow rate and pressure.
6. Use color coding to represent zones — green (normal), yellow (low pressure), red (leak detected).
7. Display overall network performance and water loss analysis using **Power BI** dashboards.

Tasks / Code Summary:

- IoT flow/pressure sensor data simulation
- MQTT → ThingSpeak integration
- AI leak detection model (Colab)
- Unity 3D urban water network visualization
- Power BI analytics dashboard

Output / Expected Result:

- Real-time detection and visualization of pipeline leaks in Unity.
- AI model accurately predicts leak-prone zones.
- Dashboard shows pressure maps and water loss insights.

Result:

Successfully implemented a water infrastructure monitoring system with AI-based leak detection and 3D visualization for proactive maintenance.

Experiment 10: Fleet Efficiency Solutions – Route Optimization and Live Vehicle Mapping

Aim:

To design an IoT and AI-based fleet monitoring system that tracks vehicles in real time, predicts optimal routes, and visualizes live vehicle movements in a 3D Unity city map.

Apparatus / Software Required:

- Unity
- Wokwi (for GPS and fuel sensor simulation)
- MQTT Broker (HiveMQ or Mosquitto)
- ThingSpeak (for cloud storage)
- Google Colab (for AI route optimization model)
- Power BI (for fleet performance dashboards)

Theory:

Fleet management systems use IoT sensors and AI to enhance transportation efficiency by tracking vehicles, optimizing routes, and reducing fuel consumption. GPS sensors provide location data, while AI models analyze traffic and performance to suggest better routes. Unity offers a 3D map-based visualization of vehicle movement, improving route visibility and operational control.

Procedure:

1. Simulate **GPS and fuel level sensors** in **Wokwi** to represent vehicle tracking data.
2. Send real-time location data through **MQTT** to **ThingSpeak** cloud channels.
3. Use **Google Colab** to create an AI model for **route optimization** based on travel distance and fuel usage.
4. Develop a **3D city map** in Unity with roads, vehicles, and routes.
5. Fetch live GPS data from ThingSpeak to animate vehicle movement in Unity.
6. Display optimized routes highlighted in green and inefficient ones in red.
7. Build **Power BI** dashboards showing total distance, fuel efficiency, and travel time.

Tasks / Code Summary:

- GPS & fuel data simulation (Wokwi)
- MQTT → ThingSpeak communication
- AI route optimization model (Colab)
- Unity 3D vehicle mapping visualization
- Power BI performance analytics

Output / Expected Result:

- Real-time visualization of fleet movement on a 3D map.
- AI suggests optimal travel routes for efficiency.
- Power BI displays fleet utilization and fuel performance metrics.

Result:

Successfully implemented a fleet management system integrating IoT tracking, AI route optimization, and Unity visualization for enhanced logistics efficiency.

Experiment 11: AI Debug Operations – Automated Issue Detection and Control Room Dashboards

Aim:

To build an AI-driven system that automatically detects, classifies, and visualizes operational issues in an industrial control room using Unity dashboards.

Apparatus / Software Required:

- Unity
- Google Colab (for NLP and AI-based log analysis)
- Power BI (for operational dashboards)
- Dataset: System or process logs (simulated data)

Theory:

In Industry 4.0 environments, vast amounts of log data are generated by connected machines and software systems. Manual log analysis is time-consuming and prone to error. Using **Natural Language Processing (NLP)** and AI models, we can automatically detect and summarize issues from log files. Unity-based control rooms provide real-time visualization of alerts, while Power BI offers analytical dashboards for system health tracking.

Procedure:

1. Collect or simulate industrial **log files** (e.g., sensor faults, connection errors, system downtime).
2. Use **Google Colab** to develop an **NLP model** that parses logs and classifies issue severity.
3. Generate automatic summaries of detected issues using AI text analysis.
4. Design a **3D control room in Unity** with multiple screens or dashboards.
5. Map issue summaries and error indicators to Unity panels dynamically.
6. Highlight critical alerts (red), medium (yellow), and normal systems (green).
7. Create **Power BI** dashboards summarizing issue trends, severity, and frequency.

Tasks / Code Summary:

- Log data preprocessing and NLP model training (Colab)
- Issue classification and summary generation
- Unity control room design with alert visualizations
- Power BI analytics for incident trends

Output / Expected Result:

- AI detects and summarizes industrial issues automatically.
- Unity control room visualizes real-time system alerts.
- Power BI shows analysis of errors, downtime, and severity distribution.

Result:

Successfully implemented an AI-powered system for automated issue detection and control room visualization in industrial environments.

Experiment 12: Mining Safety Systems – Hazard Forecasting and Underground Visualization

Aim:

To develop an IoT and AI-based mining safety monitoring system that predicts hazards such as gas leaks and vibrations, with real-time visualization of underground environments in Unity.

Apparatus / Software Required:

- Unity
- Wokwi (for IoT sensor simulation)
- MQTT Broker (HiveMQ or Mosquitto)
- ThingSpeak (for cloud storage)
- Google Colab (for hazard forecasting AI model)
- Power BI (for safety and analytics dashboards)

Theory:

Mining operations pose significant safety risks due to gas leaks, structural instability, and vibrations. IoT sensors continuously monitor air quality, gas concentration, and vibration intensity. AI models predict hazard zones based on real-time data trends. A Unity 3D digital twin of the mine enables visual navigation of tunnels and identification of danger areas for quick response and decision-making.

Procedure:

1. Simulate **gas, temperature, and vibration sensors** in **Wokwi** to represent underground monitoring.
2. Send sensor readings via **MQTT** to **ThingSpeak** cloud storage.
3. Use **Google Colab** to build a predictive model that identifies potential hazard zones using data patterns.
4. Design a **3D underground mine** in Unity with tunnels and working areas.
5. Integrate live data from ThingSpeak into Unity to visualize safety conditions dynamically.
6. Color code danger levels: green (safe), yellow (warning), red (hazard).
7. Use **Power BI** to display gas levels, vibration intensity, and hazard probability over time.

Tasks / Code Summary:

- IoT sensor simulation for gas and vibration
- MQTT → ThingSpeak data streaming
- AI hazard forecasting (Colab)
- Unity 3D underground visualization
- Power BI safety dashboards

Output / Expected Result:

- Real-time visualization of mine safety data.
- AI model predicts potential hazards and dangerous areas.
- Unity displays color-coded tunnels for safety visualization.

Result:

Successfully implemented a mining safety monitoring system that forecasts hazards and visualizes real-time underground safety conditions using IoT and AI.

Experiment 13: Smart Building HVAC – Energy Load Predictions and Dynamic Zone Mapping

Aim:

To design an IoT-enabled smart HVAC (Heating, Ventilation, and Air Conditioning) system that predicts energy load requirements using AI and visualizes dynamic temperature zones in a 3D building model using Unity.

Apparatus / Software Required:

- Unity
- Wokwi (for sensor simulation)
- MQTT Broker (HiveMQ or Mosquitto)
- ThingSpeak (for cloud data collection)
- Google Colab (for AI load prediction model)
- Power BI (for energy analytics dashboards)

Theory:

A **Smart Building HVAC system** utilizes IoT and AI to optimize energy efficiency and maintain comfortable environmental conditions. Temperature, CO₂, and humidity sensors collect real-time data, which is analyzed by AI to predict cooling or heating demand. Unity's 3D visualization allows interactive monitoring of temperature zones across different rooms or floors in a building.

Procedure:

1. Simulate **temperature, CO₂, and humidity sensors** using **Wokwi** for multiple building zones.
2. Publish sensor readings to an **MQTT broker** and log them to **ThingSpeak**.
3. Use **Google Colab** to train a **machine learning model** to predict energy load based on environmental conditions.
4. Create a **3D Unity model** of the building with various rooms and floors.
5. Integrate live data from ThingSpeak into Unity to show dynamic temperature updates.
6. Color code rooms: blue (cool), green (optimal), red (hot).
7. Use **Power BI** to create dashboards showing temperature trends, load predictions, and energy savings.

Tasks / Code Summary:

- IoT sensor simulation for HVAC parameters
- MQTT → ThingSpeak integration
- AI energy load forecasting (Colab)
- 3D building zone visualization (Unity)
- Power BI analytics for HVAC performance

Output / Expected Result:

- Real-time visualization of HVAC parameters across multiple zones.
- AI model predicts optimal energy load for each area.
- Power BI displays building energy performance and cost savings.

Result:

Successfully implemented an IoT and AI-based smart HVAC system with dynamic visualization for energy optimization and environmental control.

Experiment 14: Livestock Health Analytics – AI Disease Prediction and Farm Visualization

Aim:

To develop an IoT and AI-based livestock monitoring system that predicts animal diseases using sensor data and visualizes farm health insights in a 3D Unity environment.

Apparatus / Software Required:

- Unity
- Wokwi (for IoT sensor simulation)
- MQTT Broker (HiveMQ or Mosquitto)
- ThingSpeak (for cloud data logging)
- Google Colab (for AI disease prediction model)
- Power BI (for analytics and farm health dashboards)

Theory:

Livestock health analytics uses IoT and AI to ensure animal welfare and farm productivity. Sensors track parameters such as body temperature, heart rate, and movement. Data is sent to the cloud for AI-based analysis, which can predict possible diseases or stress conditions. Unity offers an interactive 3D visualization of the farm, helping farmers monitor herd health and respond quickly to issues.

Procedure:

1. Simulate **body temperature, pulse, and motion sensors** for animals in **Wokwi**.
2. Send real-time data via **MQTT** to **ThingSpeak** for cloud storage.
3. Train an **AI model in Google Colab** to classify animals as healthy or at risk based on sensor data.
4. Build a **3D farm environment in Unity** with animals and monitoring zones.
5. Integrate live data from ThingSpeak to display animal health conditions in real time.
6. Use color indicators: green (healthy), yellow (warning), red (disease suspected).
7. Create **Power BI dashboards** to track herd health statistics and prediction accuracy.

Tasks / Code Summary:

- IoT sensor simulation for livestock health
- MQTT → ThingSpeak integration
- AI disease prediction (Colab)
- Unity farm visualization
- Power BI farm analytics dashboard

Output / Expected Result:

- Real-time visualization of livestock health in a 3D farm.
- AI model predicts diseases or stress patterns.
- Power BI displays herd-level analytics and risk insights.

Result:

Successfully created an IoT and AI-based livestock monitoring system that predicts diseases and visualizes farm health data for efficient animal management.

Experiment 15: Intelligent Parking Systems – Occupancy Forecasting and Space Allocation

Aim:

To design an IoT-enabled smart parking system that monitors real-time parking space occupancy, forecasts availability using AI, and visualizes live slot data in Unity.

Apparatus / Software Required:

- Unity
- Wokwi (for IoT sensor simulation)
- MQTT Broker (HiveMQ or Mosquitto)
- ThingSpeak (for cloud data collection)
- Google Colab (for occupancy forecasting model)
- Power BI (for parking analytics dashboards)

Theory:

Intelligent parking systems use IoT and AI to efficiently manage vehicle parking by tracking slot availability and predicting demand. Ultrasonic or IR sensors detect vehicle presence, and AI algorithms forecast occupancy during peak hours. Unity visualizes the parking lot in 3D, helping users and administrators see slot usage and available spaces in real time.

Procedure:

1. Simulate **ultrasonic sensors** in **Wokwi** to detect car presence in each parking slot.
2. Send live occupancy data to the **ThingSpeak** cloud via **MQTT**.
3. Use **Google Colab** to train an **AI model** that predicts parking demand based on time and usage data.
4. Create a **3D parking lot in Unity** with parking slots and entry/exit gates.
5. Fetch real-time data from ThingSpeak into Unity and color slots accordingly — green (available), red (occupied).
6. Visualize AI-predicted availability during peak times.
7. Build a **Power BI dashboard** to show parking statistics, occupancy rate, and prediction accuracy.

Tasks / Code Summary:

- IoT sensor simulation for parking slot detection
- MQTT → ThingSpeak integration
- AI occupancy forecasting (Colab)
- 3D Unity parking visualization
- Power BI analytics for parking trends

Output / Expected Result:

- Real-time visualization of parking slot occupancy.
- AI model predicts parking space demand and availability.
- Power BI dashboard shows utilization and peak hour insights.

Result:

Successfully developed an IoT and AI-based intelligent parking system with real-time visualization and predictive analytics for efficient space management.

Experiment 16: Predictive Inventory Management – Demand Forecasting and Warehouse Twins

Aim:

To develop a predictive inventory management system that forecasts product demand using AI and visualizes warehouse stock levels as digital twins in Unity.

Apparatus / Software Required:

- Unity
- Google Colab (for AI demand forecasting model)
- Power BI (for inventory dashboards)
- Dataset: Product sales and stock data

Theory:

Predictive inventory management leverages AI and analytics to forecast demand, optimize stock levels, and prevent overstocking or shortages. By analyzing historical sales data, AI models can anticipate future inventory needs. Unity provides a 3D warehouse visualization, where stock changes are displayed dynamically as part of a digital twin system. This integration enables intelligent decision-making and warehouse automation.

Procedure:

1. Collect or simulate historical **sales and stock data** for multiple products.
2. In **Google Colab**, train a **machine learning model** (e.g., ARIMA, LSTM, or regression) for demand forecasting.
3. Export forecast results to a structured format for visualization.
4. Design a **3D warehouse model in Unity** with storage racks and product boxes.
5. Map the forecasted inventory data to Unity and visualize stock levels with color coding:
 - Green – Sufficient stock
 - Yellow – Low stock
 - Red – Reorder required
6. Display reorder alerts dynamically in the Unity scene.
7. Use **Power BI** to build dashboards for sales trends, product demand, and forecast accuracy.

Tasks / Code Summary:

- AI demand forecasting model (Colab)
- Data visualization integration in Unity
- Power BI dashboard for inventory and sales analytics

Output / Expected Result:

- Accurate demand forecasting using AI.
- Real-time 3D visualization of warehouse stock levels.
- Power BI dashboards displaying trends and inventory performance.

Result:

Successfully implemented a predictive inventory management system integrating AI analytics and Unity digital twins for proactive stock optimization.

Experiment 17: Fire & Gas Plant Safety – Risk Predictions and Emergency Visualization

Aim:

To design an IoT-based fire and gas detection system that predicts risk levels using AI and visualizes safety conditions and emergency zones in a 3D Unity plant environment.

Apparatus / Software Required:

- Unity
- Wokwi (for gas and flame sensor simulation)
- MQTT Broker (HiveMQ or Mosquitto)
- ThingSpeak (for cloud data storage)
- Google Colab (for AI risk prediction model)
- Power BI (for safety dashboards and analytics)

Theory:

Industrial plants require continuous monitoring of gas leaks and fire hazards to ensure workplace safety. IoT sensors detect abnormal levels of flammable gases or smoke, transmitting the data to cloud servers for analysis. AI models evaluate the probability of incidents, while Unity visualizes the plant layout and highlights hazardous zones. Power BI provides historical trends and incident predictions for preventive maintenance.

Procedure:

1. Simulate **gas and flame sensors** in **Wokwi** to detect methane, smoke, or temperature spikes.
2. Transmit real-time sensor data to **ThingSpeak** through **MQTT**.
3. Use **Google Colab** to develop an AI model that predicts risk levels based on sensor readings.
4. Create a **3D Unity plant model** showing pipelines, control rooms, and safety exits.
5. Integrate real-time data from ThingSpeak to color-code zones in Unity:
 - Green – Safe
 - Yellow – Warning
 - Red – High Risk (fire/gas detected)
6. Implement emergency indicators such as alarms or blinking lights in Unity.
7. Develop a **Power BI** dashboard to display gas concentration trends and historical safety performance.

Tasks / Code Summary:

- IoT gas and flame data simulation (Wokwi)
- MQTT → ThingSpeak data integration
- AI risk prediction (Colab)
- Unity safety visualization and alert system
- Power BI dashboard for safety monitoring

Output / Expected Result:

- Real-time visualization of gas leak and fire detection.
- AI predicts potential risk zones accurately.
- Unity displays emergency alerts and safety paths dynamically.

Result:

Successfully implemented a fire and gas plant safety monitoring system with predictive analytics, 3D visualization, and real-time emergency alerts.

Experiment 18: Precision Irrigation – AI Water Usage Forecasts and Field Visualization

Aim:

To implement an IoT-based precision irrigation system that predicts optimal water usage using AI and visualizes irrigation activities and field conditions in Unity.

Apparatus / Software Required:

- Unity
- Wokwi (for IoT sensor simulation)
- MQTT Broker (HiveMQ or Mosquitto)
- ThingSpeak (for cloud data collection)
- Google Colab (for AI water usage forecasting)
- Power BI (for irrigation efficiency dashboards)

Theory:

Precision irrigation integrates IoT sensors and AI analytics to ensure crops receive the right amount of water at the right time. Soil moisture and temperature sensors provide real-time data for analysis. AI models predict irrigation schedules and optimize water distribution, preventing wastage. Unity's 3D visualization enables real-time monitoring of irrigation systems and crop health, while Power BI presents data-driven insights for sustainable agriculture.

Procedure:

1. Simulate **soil moisture and temperature sensors** in **Wokwi** to represent various zones of a crop field.
2. Send live data via **MQTT** to **ThingSpeak** for cloud storage.
3. Train an **AI model in Google Colab** to predict irrigation needs based on soil conditions and weather trends.
4. Create a **3D Unity farm environment** with crop fields, irrigation systems, and water pumps.
5. Integrate real-time sensor data from ThingSpeak to Unity to visualize active irrigation areas.
6. Highlight zones:
 - o Green – Optimal moisture
 - o Yellow – Low moisture (needs watering)
 - o Blue – Currently irrigating
7. Develop a **Power BI** dashboard showing daily water usage, moisture trends, and AI-predicted irrigation efficiency.

Tasks / Code Summary:

- IoT soil moisture & temperature simulation (Wokwi)
- AI water usage forecasting (Colab)
- Unity 3D irrigation animation
- Power BI water management dashboard

Output / Expected Result:

- Real-time visualization of irrigation status and soil moisture levels.
- AI model accurately predicts irrigation requirements.
- Power BI dashboard displays water usage trends and efficiency reports.

Result:

Successfully created a precision irrigation system integrating IoT sensors, AI water forecasting, and Unity visualization for smart farming.

Experiment 19: Warehouse Robotics – Path Optimization and Fleet Coordination Analytics

Aim:

To develop an AI-based warehouse robotic system that optimizes robot navigation paths, coordinates multiple fleets, and visualizes real-time operations in Unity.

Apparatus / Software Required:

- Unity
- Google Colab (for AI path optimization model)
- Power BI (for performance analytics dashboards)
- Dataset: Warehouse layout and robot path logs

Theory:

Warehouse automation is a key component of Industry 4.0, where multiple robots operate simultaneously for picking, placing, and transporting goods. AI algorithms optimize paths to minimize travel time and avoid collisions. Fleet coordination ensures multiple robots work efficiently without overlap. Unity's 3D environment provides an interactive visualization of robot movements and warehouse performance.

Procedure:

1. Create or import a **warehouse layout** with shelves, aisles, and pickup zones in **Unity**.
2. Simulate multiple **robots (AGVs or AMRs)** moving between racks using navigation paths.
3. Collect robot position data and logs for path optimization training.
4. In **Google Colab**, train an **AI optimization model** using algorithms like A*, Dijkstra's, or reinforcement learning to minimize travel distance.
5. Integrate optimized paths into **Unity**, allowing robots to move dynamically according to AI outputs.
6. Highlight busy or congested routes with color indicators.
7. Use **Power BI** to display metrics such as travel efficiency, idle time, and collision avoidance statistics.

Tasks / Code Summary:

- AI model for path optimization (Colab)
- Unity 3D simulation for robot movement
- Power BI analytics for fleet coordination and efficiency

Output / Expected Result:

- Robots navigate efficiently in a 3D warehouse without collisions.
- AI optimizes paths for reduced travel time and better coordination.
- Power BI provides visual analytics for warehouse robot operations.

Result:

Successfully created a warehouse robotics system integrating AI path optimization, 3D visualization, and fleet coordination analytics.

Experiment 20: AI-Driven Supply Chains – Predictive Insights and Risk Visualization

Aim:

To create an AI-powered supply chain monitoring system that predicts risks and delays, and visualizes global logistics flows in a 3D environment using Unity.

Apparatus / Software Required:

- Unity
- Google Colab (for AI risk prediction and forecasting model)
- Power BI (for supply chain analytics)
- Dataset: Supplier, shipment, and logistics data

Theory:

An AI-driven supply chain integrates IoT data, predictive analytics, and visualization tools to manage complex logistics operations efficiently. AI models forecast potential disruptions, such as delays or shortages, by analyzing shipment, route, and supplier data. Unity offers an immersive visualization of the supply network, while Power BI provides insight into supply risks, delivery performance, and predictive trends for proactive decision-making.

Procedure:

1. Gather or simulate **supplier, shipment, and logistics data** (e.g., routes, time, distance, and delivery status).
2. Train a **machine learning model in Google Colab** to predict delays, bottlenecks, and supplier risks.
3. Design a **3D Unity globe** with key trade routes, suppliers, and destination nodes.
4. Integrate AI-predicted results into Unity, mapping supply chain paths dynamically.
5. Color code shipment status:
 - Green – On Time
 - Yellow – At Risk
 - Red – Delayed or Disrupted
6. Add interactive controls to visualize shipments by region, product, or risk level.
7. Use **Power BI** to build dashboards for supplier performance, delivery times, and risk trends.

Tasks / Code Summary:

- AI model for supply chain risk prediction (Colab)
- 3D Unity visualization for global logistics
- Power BI dashboard for predictive insights

Output / Expected Result:

- Real-time visualization of global supply chains in Unity.
- AI predicts potential risks and delays accurately.
- Power BI displays key logistics metrics and performance trends.

Result:

Successfully designed an AI-driven supply chain management system with predictive risk analysis and 3D visualization for global logistics optimization.